

Comprehensive Technical Assessment and Pipeline Blueprint for IoT Intrusion Detection via the CICIoT2023 Benchmark

1. Dataset Analysis

The Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick (UnB) introduced the CICIoT2023 dataset to establish a modern benchmark for Network Intrusion Detection Systems (NIDS) operating within Internet of Things (IoT) topologies¹. The raw capture consists of approximately 548 gigabytes of packet capture (PCAP) files¹, while the pre-extracted tabular representation occupies 27 gigabytes in comma-separated values (CSV) format². The dataset is publicly available through the official CIC dataset portal at <https://www.unb.ca/cic/datasets/iotdataset-2023.html> under a Creative Commons Attribution 4.0 International license¹.

The testbed architecture simulates a smart home deployment comprising 105 physical IoT endpoints¹. The victim subnet includes 67 diverse physical endpoints—such as smart cameras, smart speakers, home automation gateways, smart bulbs, microcontrollers, and smart plugs—alongside 38 Zigbee and Z-Wave peripheral sensors attached via dedicated control hubs¹. A key architectural distinction of CICIoT2023 is that attack campaigns were executed directly from 7 compromised Raspberry Pi 4 microcontrollers embedded within the target network, rather than from external desktop computers¹. Traffic capture was executed using a dedicated hardware Gigamon Network Tap (G-TAP A-Tx) positioned between the Cisco Catalyst 3850 switch infrastructure and two network monitoring engines running Wireshark, capturing full-duplex traffic without packet drop or latency injection¹.

The benchmark dataset contains 46,686,579 instances categorized into 34 target labels, comprising 33 distinct attack profiles and 1 legitimate baseline capture¹. Benign operations were captured over 16 consecutive hours, encompassing both background idle states and manual device interactions¹. The label taxonomy is evaluated at three distinct levels of abstraction: a 2-class binary formulation (Malicious vs. Benign), an 8-class aggregated functional category taxonomy, and a 34-class fine-grained attack profile taxonomy¹.

Attack Category	Specific Attack Profiles	Total Instance Count	Dataset Share (%)
DDoS	ICMP Flood, UDP	33,984,560	72.79%

	Flood, TCP Flood, PSHACK Flood, SYN Flood, RSTFIN Flood, SynonymousIP Flood, ICMP Fragmentation, UDP Fragmentation, ACK Fragmentation, HTTP Flood, SlowLoris		
DoS	UDP Flood, TCP Flood, SYN Flood, HTTP Flood	8,090,738	17.33%
Mirai	Greeth Flood, UDPPlain, GREIP Flood	2,634,124	5.64%
Benign	Normal IoT traffic (idle states, sensor polling, voice commands, video streams)	1,098,195	2.35%
Spoofing	MITM ARP Spoofing, DNS Spoofing	486,504	1.04%
Reconnaissance	Host Discovery, OS Scan, Port Scan, Vulnerability Scan, Ping Sweep	354,565	0.76%
Web-based	Browser Hijacking, Command Injection, SQL Injection, XSS,	24,829	0.05%

	Backdoor Malware, Uploading Attack		
Brute Force	Dictionary Brute Force	13,064	0.03%

The dataset provides 47 extracted flow attributes². Excluding the sorting timestamp (ts), 46 features remain for model development¹. These features capture flow duration, protocol indicators, frame transmission rates, TCP control flags, application-layer protocols, statistical packet length distributions, inter-arrival times (IAT), flow size metrics, and spatial directional properties¹.

Feature Name	Type	Description
flow_duration	Continuous	Total temporal duration of the packet flow sequence ¹
Header_Length	Continuous	Cumulative length of packet headers within the flow ¹
Protocol_Type	Categorical	Numeric identifier for transport/network layer protocol (TCP, UDP, ICMP, IGMP) ¹
Duration	Continuous	Time-to-Live (TTL) header value ¹
Rate, Srate, Drate	Continuous	Total, outbound (source), and inbound (destination) packet transmission rates ¹
fin_flag_number, syn_flag_number, rst_flag_number, psh_flag_number, ack_flag_number, ece_flag_number,	Binary Flags	Individual TCP flag presence indicators per packet frame ¹

cwr_flag_number		
ack_count, syn_count, fin_count, urg_count, rst_count	Discrete	Cumulative counts of specific TCP flags within the observed flow window ¹
HTTP, HTTPS, DNS, Telnet, SMTP, SSH, IRC, DHCP, ARP, ICMP, IPv, LLC	Binary Flags	One-hot application, transport, network, and link layer protocol indicators ¹
Tot sum, Min, Max, AVG, Std, Tot size	Continuous	Statistical metrics summarizing packet length distributions within the flow ¹
IAT	Continuous	Inter-arrival time difference relative to the preceding packet frame ¹
Number	Discrete	Count of packets tracked within the current sliding window ¹
Magnitude, Radius, Covariance, Variance, Weight	Continuous	Directional spatial metrics measuring packet size variance between incoming and outgoing streams ¹

Critiques in the network security literature highlight several methodological limitations and structural biases in the CICIoT2023 dataset⁷:

- **Severe Class Imbalance:** Volumetric flooding vectors (DDoS and DoS) represent over 90.1% of the total records, whereas targeted threats such as Web-based exploits (0.05%) and Brute Force attempts (0.03%) are severely under-represented¹. Unweighted classifiers trained on raw distributions exhibit significant bias toward majority classes, resulting in low recall for rare attack types⁷.
- **High Feature Redundancy:** Transmission rates (Rate and Srate) and frame size totals (Tot sum and Tot size) display near-perfect linear correlation, introducing redundant dimensions that increase training overhead without improving predictive performance⁴.
- **Window Aggregation Discrepancies:** The extraction pipeline aggregates flows using a 100-packet window for high-volume flooding traffic and a 10-packet window for low-rate vectors¹. This dual-windowing approach introduces artificial variance artifacts that models can exploit as shortcuts during training¹.

- **Synthetic Scripting Bias:** Scripted attack executions from microcontrollers fail to capture the stochastic packet drop, network jitter, and complex background noise typical of enterprise IoT deployments⁸.

Benchmark Dataset	Creation Year	Physical Devices	Attack Classes	Total Instances	Feature Count	Target Network Environment
CICIoT2023	2023	105	34	46,686,579	47	Heterogeneous Consumer & Smart Home IoT ¹
Edge-IIoTSet	2022	>10	14	>20,000,000	61	Edge and Industrial IoT (IIoT) infrastructure ¹
TON_IoT	2020	Heterogeneous	10	~22,000,000	44	Combined Telemetry, OS, and Network captures ¹
BoT-IoT	2019	5	5	>72,000,000	29	Botnet flooding and scan vectors ¹
IoT-23	2020	3 real / 20 emulated	11	>30,000,000	23	Real malware execution on IoT hardware ¹

N-BaloT	2018	9	2	7,062,606	115	Deep Autoencoder botnet anomaly detection ¹
CICIDS2017	2017	Standard IT	8	2,830,743	78	Legacy enterprise IT infrastructure ²

Compared to legacy datasets like CICIDS2017 or UNSW-NB15, CICIoT2023 provides better coverage of native IoT protocols, hardware micro-renderers, and modern Mirai botnet variations¹.

2. Data Preprocessing

The dataset pipeline transforms raw binary network traffic into standardized tabular format through a structured multi-stage workflow¹. Large PCAP captures (548 GB) are segmented into 10 MB sub-files using tcpdump to enable parallel processing¹. A custom feature extraction pipeline powered by the Python dpkt library parses packet headers to compute statistical flow properties¹. Incomplete packet headers containing null values are discarded¹. The sorting timestamp attribute (ts) is explicitly removed prior to model training to prevent models from learning temporal artifacts instead of structural network behaviors¹.

Traffic aggregation uses sliding packet windows to compute mean feature values: low-rate vectors (Web attacks, Brute Force, Reconnaissance, Spoofing, and Benign operations) are grouped into 10-packet windows, while high-volume vectors (DDoS, DoS, Mirai) use 100-packet windows¹. The resulting sub-files are merged into a unified dataset using PySpark¹. Continuous numerical attributes are normalized using StandardScaler to enforce zero mean and unit variance¹:

$$z = \frac{x - \mu}{\sigma}$$

While the original study retained all 46 predictors¹, subsequent research has investigated feature selection methods to improve computational efficiency:

- **Mutual Information (MI) Selection:** Measures non-linear dependencies between input features and target labels⁹. This approach reduces the feature space from 46 to 18–20 attributes while preserving over 99% of baseline classification accuracy⁹.
- **Gain-Based Tree Selection:** Ranks features by mean impurity reduction across Random

Forest or XGBoost models⁴. Pruning low-gain attributes (such as rarely set TCP flags like ECE or CWR) cuts inference latency by 30% to 51%¹.

- **Autoencoder Latent Dimensionality Reduction:** Uses bottleneck neural layers to extract compact non-linear representations for deep learning models¹⁵.

The baseline paper did not apply class rebalancing, evaluating performance on the unweighted imbalanced distributions¹. To address classification bias on minority classes, subsequent studies have used three main rebalancing strategies:

- **Stratified Undersampling:** Caps majority DDoS and DoS records while enforcing a minimum retention floor (e.g., 1,000 instances per class) for minority vectors⁴. This reduces total dataset size to 4–5 million instances, lowering compute requirements while preserving fine-grained 34-class evaluation integrity⁴.
- **Synthetic Minority Oversampling Technique (SMOTE):** Interpolates synthetic feature vectors within minority classes (Web-based, Brute Force) to balance class distributions prior to model training⁷.
- **Cost-Sensitive Loss Weighting & Focal Loss:** Dynamically adjusts the loss function during training to penalize misclassifications on rare attack classes without altering dataset instance counts¹⁹.

The baseline study used an 80/20 train/test split via PySpark stratified sampling, maintaining proportional class representation across both partitions¹.

3. Modeling Methodology

The baseline benchmark evaluated five machine learning models representing linear models, ensemble techniques, and deep learning architectures¹:

- **Logistic Regression:** A linear baseline using multinomial log-loss optimization to evaluate linear separability across flow features¹.
- **Perceptron:** A single-layer neural network serving as a non-ensemble linear threshold benchmark¹.
- **AdaBoost:** An adaptive boosting ensemble that sequentially trains decision stumps, adjusting sample weights to focus on previously misclassified instances¹.
- **Random Forest:** A bagging ensemble of decision trees constructed via randomized feature subsampling¹. Tree ensembles effectively capture non-linear feature interactions without requiring strict distributional assumptions¹.
- **Deep Neural Network (DNN):** A fully connected multi-layer perceptron architecture¹. Deep architectures leverage multi-layer non-linear transformations to learn complex representations across large tabular datasets¹.

In broader literature using CICIoT2023, researchers have introduced advanced architectures to improve detection performance:

- **Gradient Boosted Decision Trees (XGBoost, LightGBM, CatBoost):** Tree-based gradient boosting models consistently outperform deep neural networks on tabular flow datasets⁴. LightGBM uses histogram-based feature binning and leaf-wise tree growth to

achieve fast training speeds and high macro F1-scores across 34-class tasks⁴.

- **1D-CNN + LSTM Hybrid Architectures:** One-dimensional Convolutional Neural Networks (1D-CNN) extract spatial correlations across packet attributes, while Long Short-Term Memory (LSTM) layers capture temporal patterns across successive network flows¹³. This hybrid design improves detection of multi-stage attacks such as Mirai botnet campaigns¹⁵.
- **Hierarchical Two-Stage Frameworks:** Stage 1 uses a lightweight Decision Tree to rapidly filter high-volume DDoS/DoS traffic⁹. Remaining traffic is passed to a Stage 2 multi-class ensemble (e.g., Random Forest or XGBoost) to classify complex low-rate threats, reducing overall processing latency by up to 64.5%⁹.

4. Training Procedure

The benchmark experiments were implemented using PySpark for distributed data processing alongside Python machine learning libraries¹. Models were trained on the 80% training partition following continuous feature standardization via StandardScaler¹.

Deep Neural Network configurations used dropout layers and validation-driven early stopping to prevent overfitting¹. Random Forest models used Gini impurity to evaluate node splits across randomized feature subsets¹. Hyperparameter selection in the baseline paper relied on manual empirical tuning¹. Subsequent literature applies systematic optimization techniques:

- **Grid Search:** Evaluates exhaustive parameter combinations across pre-defined search grids, commonly used to tune Random Forest tree depth and estimator counts⁴.
- **Random Search:** Samples hyperparameter spaces randomly to efficiently configure gradient boosted decision trees¹².
- **Bayesian Optimization:** Uses Gaussian Process surrogate models to iteratively select hyperparameter combinations that maximize validation macro F1-scores²⁷.

5. Evaluation

Model evaluation uses standard binary and multi-class classification metrics¹:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The baseline study evaluated performance across three task granularities: binary (2 classes), functional categories (8 classes), and individual attack profiles (34 classes)¹.

Task Granularity	Model Architecture	Accuracy	Recall	Precision	F1-Score
2 Classes	Random Forest	0.9968	0.9652	0.9654	0.9653
	Deep Neural Network	0.9944	0.9333	0.9476	0.9403
	AdaBoost	0.9959	0.9473	0.9656	0.9563
	Logistic Regression	0.9890	0.8904	0.8632	0.8763
	Perceptron	0.9818	0.7970	0.8254	0.8105
8 Classes	Random Forest	0.9944	0.9100	0.7054	0.7193
	Deep Neural Network	0.9911	0.9066	0.6794	0.6973
	Perceptron	0.8663	0.6591	0.5239	0.5551
	Logistic Regression	0.8317	0.6961	0.5124	0.5394
	AdaBoost	0.3514	0.4878	0.4649	0.3687
34 Classes	Random Forest	0.9916	0.8316	0.7045	0.7140
	Deep Neural	0.9861	0.7319	0.6653	0.6723

	Network				
	Logistic Regression	0.8023	0.5952	0.4868	0.4939
	AdaBoost	0.6079	0.6077	0.4796	0.4735
	Perceptron	0.8196	0.5075	0.4546	0.4473

Random Forest and Deep Neural Networks achieve high overall accuracy (>98.6%) across all three task granularities¹. However, F1-scores drop significantly on multi-class tasks¹. This gap illustrates the impact of class imbalance: overall accuracy remains high because models correctly classify the overwhelming majority of volumetric DDoS samples, masking poor performance on minority attack types¹.

Confusion matrix analyses from the baseline paper reveal distinct per-class performance patterns¹:

- **High-Volume Flooding Classes:** DDoS, DoS, and Mirai variants achieve classification accuracies above 99.5% due to their distinctive volumetric flow profiles¹.
- **Low-Volume Targeted Classes:** Web-based attacks (SQL Injection, XSS, Command Injection) and Dictionary Brute Force attempts show high misclassification rates, often being confused with Benign traffic, Reconnaissance scans, or Spoofing attempts due to shared statistical attributes¹.

Ablation studies in subsequent research show that feature selection reduces the feature space from 46 to 18–23 dimensions, cutting training time by 33% to 40% and reducing per-flow inference latency from ~75 microseconds to 26.66 microseconds with minimal impact on accuracy (within $\pm 0.014\%$)⁴. Applying SMOTE or cost-sensitive focal loss significantly improves minority class recall, raising overall macro F1-scores on 34-class tasks from ~71.4% to over 97%⁴.

6. Scope Recommendation for My Capstone

Training models on the complete CICIoT2023 dataset (46.6+ million records, ~27 GB CSV) presents substantial computational challenges for a standard capstone project. To design a defensible research pipeline within typical compute constraints, the project scope should focus on targeted subsampling, feature selection, and model comparison.

Subsampling Strategy

Extract a stratified working sample of 2,000,000 to 4,000,000 instances from the processed CSV dataset⁴. Apply stratified undersampling to majority DDoS/DoS classes while retaining all available samples for low-frequency categories (Web-based, Brute Force), enforcing a

minimum retention floor of 1,000 instances per class⁴. This maintains representation for rare attack types while keeping memory requirements manageable on standard workstations⁴.

Modeling Scope

Implement and compare four core model families:

- **Baseline Model:** Random Forest or Decision Tree¹.
- **Gradient Boosted Trees:** LightGBM and XGBoost⁴.
- **Deep Learning Baseline:** Fully connected Deep Neural Network (DNN)¹.
- **Advanced Deep Learning Model:** 1D-CNN + LSTM hybrid architecture²⁴.

Core Research Contributions

Evaluate the model development pipeline across two primary experimental axes:

- **Feature Dimensionality:** Compare model performance using the full set of 46 features against a reduced set of 18–20 features selected via Gain-Based or Mutual Information ranking⁴. Measure the impact on both classification metrics and inference latency⁴.
- **Class Imbalance Mitigation:** Compare baseline (unbalanced) training against stratified undersampling, SMOTE oversampling, and cost-sensitive class weighting⁴. Evaluate performance gains using Macro F1-score, Precision-Recall AUC, and per-class Recall metrics⁴.

Methodological Limitations and Reproducibility Mitigations

- **Raw PCAP Processing Bottlenecks:** Parsing 548 GB of raw PCAP files requires significant compute and storage infrastructure¹. *Mitigation:* Use the pre-aggregated CSV feature sub-files provided by the CIC repository, bypassing raw PCAP parsing entirely¹.
- **Accuracy Metric Distortion:** Overall accuracy metrics are inflated by majority flood classes, masking poor performance on minority attack types¹. *Mitigation:* Adopt Macro F1-score, Precision-Recall curves, and per-class confusion matrices as primary evaluation metrics⁴.
- **Temporal Data Leakage:** Random packet-level splits across sliding windows cause data leakage between training and testing partitions¹. *Mitigation:* Perform train/test splits at the capture-session level prior to window aggregation, ensuring strict temporal separation between training and evaluation subsets¹.

Works cited

1. sensors-23-05941-v2.pdf
2. Overview of the CICIoT2023 Dataset for Internet of Things Intrusion, <https://map.researchcommons.org/cgi/viewcontent.cgi?article=1050&context=mjbd>
3. CIC IoT dataset 2023 - University of New Brunswick, <https://www.unb.ca/cic/datasets/iotdataset-2023.html>
4. An optimized gradient boosting framework for IoT intrusion detection,

- https://www.researchgate.net/publication/403725452_An_optimized_gradient_boosting_framework_for_IIoT_intrusion_detection_a_comprehensive_evaluation_on_the_CICIoT2023_dataset
5. CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale, <https://www.mdpi.com/1424-8220/23/13/5941>
 6. CICIoT2023 - Kaggle, <https://www.kaggle.com/datasets/himadri07/ciciot2023>
 7. (PDF) Overview of the CICIoT2023 Dataset for Internet of Things, https://www.researchgate.net/publication/392610140_Overview_of_the_CICIoT2023_Dataset_for_Internet_of_Things_Intrusion_Detection_Systems
 8. A dataset-centric review of IoT and IIoT intrusion detection - Frontiers, <https://www.frontiersin.org/journals/big-data/articles/10.3389/fdata.2026.1878260/full>
 9. A Lightweight Two-Stage Intrusion Detection Framework Optimized, <https://www.techscience.com/cmc/v87n3/66938/html>
 10. Improving IoT Intrusion Detection Through SMOTE-Based ... - arXiv, <https://arxiv.org/html/2606.00161v1>
 11. Comparative Analysis of Machine Learning Classifiers for Detecting, <https://www.jcibi.org/index.php/Main/article/download/1352/856>
 12. BINARY AND MULTI-CLASS INTRUSION DETECTION IN IOT, <https://arxiv.org/pdf/2503.22684>
 13. Ensemble-Based Intrusion Detection for IoT Networks Using the, <https://jisem-journal.com/index.php/journal/article/download/3407/1475/5588>
 14. Overview of the CICIoT2023 Dataset for Internet of Things Intrusion, <https://map.researchcommons.org/mjbd/vol5/iss1/4/>
 15. Intelligent Intrusion Detection System Against Various Attacks Based, <https://pmc.ncbi.nlm.nih.gov/articles/PMC11768945/>
 16. DDoS classification of network traffic in software defined ... - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12334730/>
 17. IIDS: Design of Intelligent Intrusion Detection System for Internet-of, <https://arxiv.org/html/2308.00943v1>
 18. Improving IoT Intrusion Detection Through SMOTE-Based ... - arXiv, <https://arxiv.org/pdf/2606.00161>
 19. A Lightweight Network Intrusion Detection System Based on, <https://computerfraudsecurity.com/index.php/journal/article/download/584/405>
 20. Original vs. undersampled binary class distribution of the, https://www.researchgate.net/figure/Original-vs-undersampled-binary-class-distribution-of-the-CICIoT2023-dataset_fig5_403725452
 21. Intrusion Detection Framework for Internet of Things with Rule, <https://www.mdpi.com/1424-8220/25/6/1845>
 22. Features importance across three tasks (LightGBM vs. XGBoost)., https://www.researchgate.net/figure/Features-importance-across-three-tasks-LightGBM-vs-XGBoost_fig9_403725452
 23. XAI-Driven Intrusion Detection for Internet of Things Networks - MDPI, <https://www.mdpi.com/1424-8220/26/17/5437>
 24. IoT-Based Intrusion Detection System Using New Hybrid Deep,

<https://www.mdpi.com/2079-9292/13/6/1053>

25. Deep Learning-Based Classification of IoT DDoS Attacks Using,
https://www.researchgate.net/publication/396413126_Deep_Learning-Based_Classification_of_IoT_DDoS_Attacks_Using_CNN-LSTM
26. Deep learning based intelligent IDS for Large-scale IoT networks,
<https://arxiv.org/html/2603.16342v2>
27. CICIOT2023: A real-time dataset and benchmark for large-scale,
https://www.researchgate.net/publication/370611316_CICIOT2023_A_real-time_dataset_and_benchmark_for_large-scale_attacks_in_IoT_environment
28. Network intrusion detection model using wrapper based feature,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12328701/>